Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2026-27

Class:	**TE-AI&DS**	**Semester:**	**V**
Course Code:	**2015111**	**Course Name:**	**Statistics for Machine Learning and Data Science Lab**

Name of Student:	**Divya S. Konar**
Roll No. :	**20**
Experiment No.:	**8**
Title of the Experiment:	**Apply Clustering and Dimensionality Reduction Techniques for Pattern Discovery**
Date of Performance:	**09/09/20**
Date of Submission:	**23/09/20**

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : **Dr. Tatwadarshi P.N.**

Signature :

Date :

Aim: To apply clustering and dimensionality reduction techniques to the Pima Indians Diabetes Dataset to discover hidden patterns and visualize relationships among patients.

Objectives

After completing this experiment, the student will be able to:

1. Apply clustering and dimensionality reduction techniques to discover patterns in a high-dimensional dataset.
2. Interpret the resulting clusters and reduced-dimensional representations using suitable evaluation measures and visualizations.

Tools Required

- Python 3.x
- Google Colab / Jupyter Notebook
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn

Dataset Details

Dataset: Pima Indians Diabetes Dataset

The dataset contains medical information for **768 female patients.** The Outcome variable indicates diabetes status.

The Outcome variable may be excluded during unsupervised learning and used later to compare the discovered clusters with the known diabetes categories.

Procedure

1. Load the Pima Indians Diabetes Dataset.
2. Select relevant numerical features for unsupervised analysis.

3. Handle invalid or missing values.
4. Standardize the selected features.
5. Apply K-Means clustering.
6. Select a suitable number of clusters using the Elbow Method.
7. Evaluate the clustering using the Silhouette Score.
8. Apply PCA to reduce the dimensionality of the dataset.
9. Visualize the observations in the reduced two-dimensional space.
10. Analyze the discovered clusters and interpret the results.

Description of the Experiment

Unsupervised learning identifies patterns in data without using a target variable during model development.

The experiment follows: High-Dimensional Dataset, Data Preprocessing, Feature Standardization, Clustering, Cluster Evaluation, Dimensionality Reduction, Visualization and Interpretation

Clustering groups similar observations, while dimensionality reduction represents high-dimensional data using fewer dimensions for visualization and analysis.

Detailed Description of Statistical Methods

K-Means Clustering

K-Means divides observations into *k*clusters by minimizing the distance between observations and their assigned cluster centroid.

The objective function is:

$$J = \sum_{j=1}^{k} \sum_{x_i \in C_j} \| x_i - \mu_j \|^2$$

where:

- C_j = cluster j

- μ_j = centroid of cluster j

Elbow Method

The Elbow Method evaluates different values of $k$using within-cluster sum of squares.

The value of $k$is selected by identifying the point where adding more clusters provides diminishing improvement.

Silhouette Score

The Silhouette Score evaluates how well an observation fits within its assigned cluster.

$$s = \frac{b-a}{max(a,b)}$$

where:

- a= average distance within the same cluster
- b= average distance to the nearest other cluster

A higher score generally indicates better-defined clusters.

Principal Component Analysis

PCA transforms correlated variables into a smaller number of uncorrelated principal components.

The principal components are obtained from the covariance matrix using eigenvalues and eigenvectors.

The first principal components capture the maximum possible variance in the data.

PCA can be used to represent the dataset in two dimensions for visualization.

Output :

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA

# Load dataset
url = "https://raw.githubusercontent.com/npradaschnor/Pima-Indians-Diabetes-Dataset/master/diabetes.csv"

df = pd.read_csv(url)

# Display first 5 rows
df.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

```
# Select features for clustering
features = [
    'Pregnancies',
    'Glucose',
    'BloodPressure',
    'SkinThickness',
    'Insulin',
    'BMI',
    'DiabetesPedigreeFunction',
    'Age'
]

X = df[features].copy()

# Check missing values
print("Missing values:")
print(X.isnull().sum())

# Check zero values
print("\nZero values:")
print((X == 0).sum())
```

```
Missing values:
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
```

[ ]

```
# Replace invalid zero values with NaN
invalid_columns = [
    'Glucose',
    'BloodPressure',
    'SkinThickness',
    'Insulin',
    'BMI'
]

X[invalid_columns] = X[invalid_columns].replace(0, np.nan)

# Replace missing values with median
X = X.fillna(X.median())

# Check again
print("Missing values after treatment:")
print(X.isnull().sum())

print("\nFirst 5 rows after preprocessing:")
print(X.head())
```

```
Missing values after treatment:
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
dtype: int64

First 5 rows after preprocessing:
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6    148.0           72.0           35.0    125.0  33.6
1            1     85.0           66.0           29.0    125.0  26.6
2            8    183.0           64.0           29.0    125.0  23.3
3            1     89.0           66.0           23.0     94.0  28.1
4            0    137.0           40.0           35.0    168.0  43.1

   DiabetesPedigreeFunction  Age
0                     0.627   50
1                     0.351   31
2                     0.672   32
3                     0.167   21
4                     2.288   33
```

```
# Standardize the features
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

# Convert back to DataFrame for easy viewing
X_scaled = pd.DataFrame(X_scaled, columns=features)

# Display first 5 standardized rows
X_scaled.head()
```

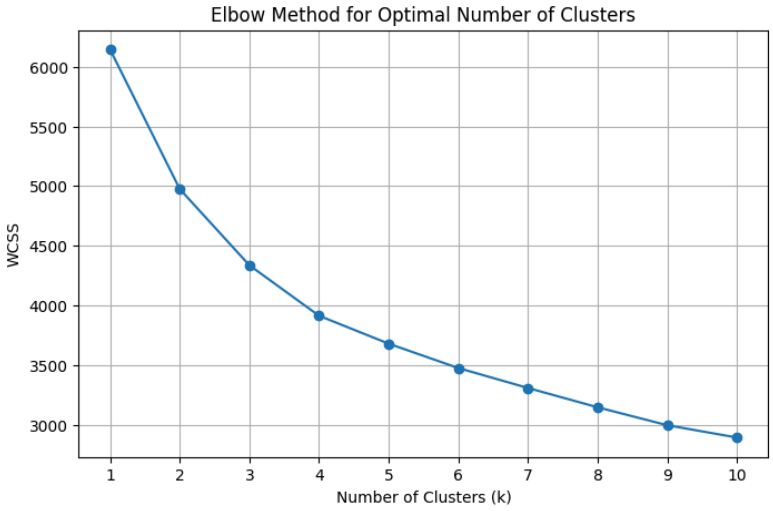
	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
0	0.639947	0.866045	-0.031990	0.670643	-0.181541	0.166619	0.468492	1.425995
1	-0.844885	-1.205066	-0.528319	-0.012301	-0.181541	-0.852200	-0.365061	-0.190672
2	1.233880	2.016662	-0.693761	-0.012301	-0.181541	-1.332500	0.604397	-0.105584
3	-0.844885	-1.073567	-0.528319	-0.695245	-0.540642	-0.633881	-0.920763	-1.041549
4	-1.141852	0.504422	-2.679076	0.670643	0.316566	1.549303	5.484909	-0.020496

```
# Calculate WCSS for different numbers of clusters
wcss = []

for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

# Plot Elbow Curve
plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), wcss, marker='o')

plt.xlabel("Number of Clusters (k)")
plt.ylabel("WCSS")
plt.title("Elbow Method for Optimal Number of Clusters")
plt.xticks(range(1, 11))
plt.grid(True)
plt.show()
```



```
# Apply K-Means clustering
kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)

# Fit the model and assign clusters
clusters = kmeans.fit_predict(X_scaled)

# Add cluster labels to the original dataframe
df['Cluster'] = clusters

# Display cluster assignments
print("Cluster assignments:")
print(df[['Outcome', 'Cluster']].head(10))
```

```
Cluster assignments:
   Outcome  Cluster
0        1        0
1        0        1
2        1        0
3        0        1
4        1        1
5        0        1
6        1        1
7        0        0
8        1        0
9        1        0
```

```
# Count observations in each cluster
print("Number of patients in each cluster:")
print(df['Cluster'].value_counts().sort_index())
```

```
Number of patients in each cluster:
Cluster
0    351
1    417
Name: count, dtype: int64
```

```
# Calculate Silhouette Score
sil_score = silhouette_score(X_scaled, clusters)

print("Silhouette Score:", sil_score)
```

```
Silhouette Score: 0.19627861483317974
```

```
# Apply PCA to reduce the data to 2 dimensions
pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)

# Create a DataFrame for PCA results
pca_df = pd.DataFrame(
    X_pca,
    columns=['Principal Component 1', 'Principal Component 2']
)

# Add cluster labels
pca_df['Cluster'] = clusters

# Display first 5 rows
pca_df.head()
```

	Principal Component 1	Principal Component 2	Cluster
0	1.449481	0.659769	0
1	-1.498835	-0.076049	1
2	0.423246	0.761327	0
3	-2.152468	-0.166717	1
4	0.697836	-3.548952	1

```
# Explained variance by each principal component
print("Explained variance ratio:")
print(pca.explained_variance_ratio_)

print("\nTotal variance explained by 2 components:")
print(pca.explained_variance_ratio_.sum())
```

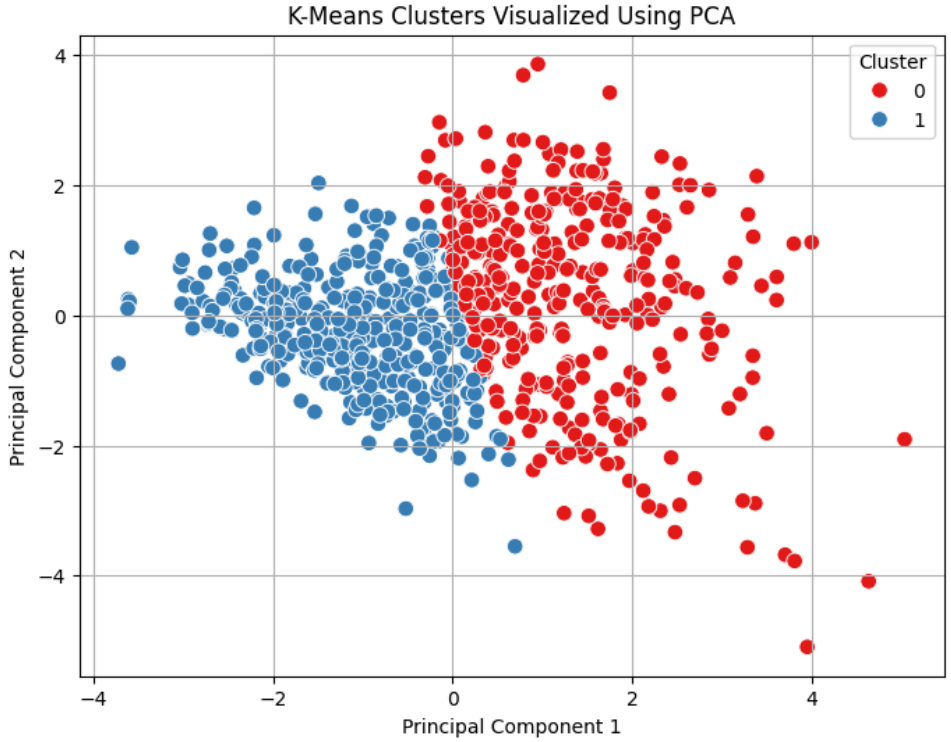
```
Explained variance ratio:
[0.28537316 0.18694722]

Total variance explained by 2 components:
0.4723203741075307
```

```
plt.figure(figsize=(8, 6))

sns.scatterplot(
    data=pca_df,
    x='Principal Component 1',
    y='Principal Component 2',
    hue='Cluster',
    palette='Set1',
    s=70
)

plt.title("K-Means Clusters Visualized Using PCA")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend(title="Cluster")
plt.grid(True)
plt.show()
```




```
[ ]
# Compare clusters with actual diabetes outcome
cluster_outcome = pd.crosstab(
    df['Cluster'],
    df['Outcome']
)

print("Cluster vs Diabetes Outcome:")
print(cluster_outcome)
```

```
Cluster vs Diabetes Outcome:
Outcome    0    1
Cluster
0        156  195
1        344   73
```

```
[ ]
# Percentage distribution of diabetes outcome within each cluster
cluster_percentage = pd.crosstab(
    df['Cluster'],
    df['Outcome'],
    normalize='index'
) * 100

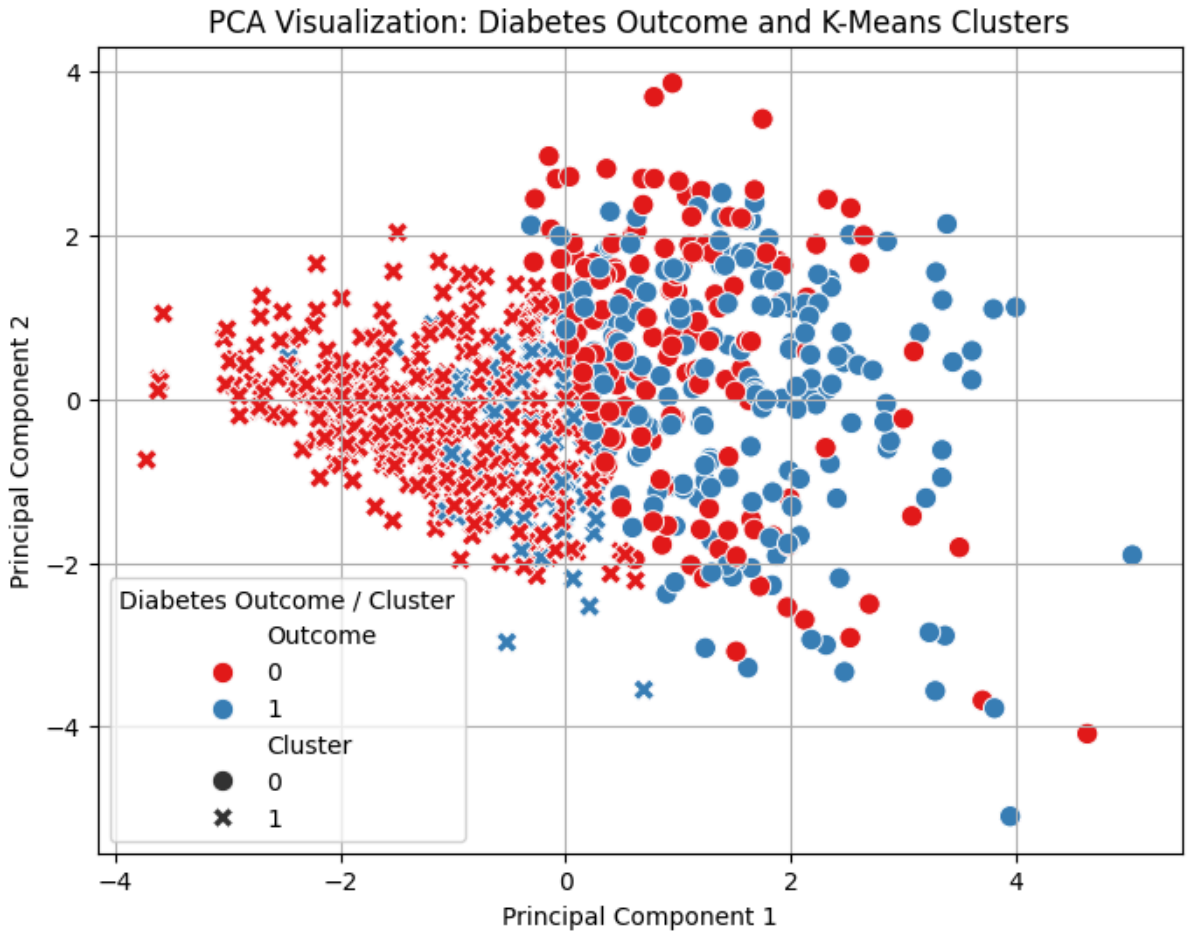
print("Percentage distribution:")
print(cluster_percentage.round(2))
```

```
Percentage distribution:
Outcome      0      1
Cluster
0        44.44  55.56
1        82.49  17.51
```

```
plt.figure(figsize=(8, 6))

sns.scatterplot(
    data=pca_df,
    x='Principal Component 1',
    y='Principal Component 2',
    hue=df['Outcome'],
    style=df['Cluster'],
    palette='Set1',
    s=80
)

plt.title("PCA Visualization: Diabetes Outcome and K-Means Clusters")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend(title="Diabetes Outcome / Cluster")
plt.grid(True)
plt.show()
```



```
# Calculate mean values of features for each cluster
cluster_means = df.groupby('Cluster')[features].mean()

print("Mean feature values for each cluster:")
print(cluster_means.round(2))
```

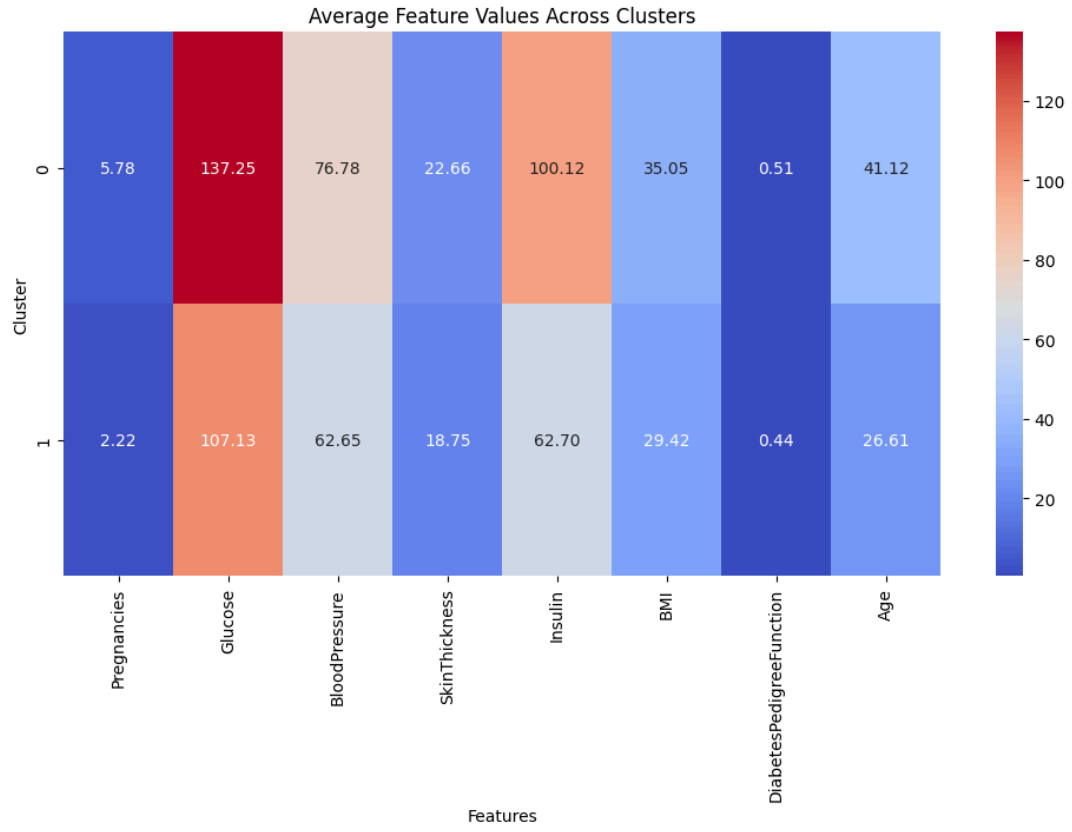
```
Mean feature values for each cluster:
         Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin    BMI  \
Cluster
0               5.78   137.25          76.78          22.66   100.12  35.05
1               2.22   107.13          62.65          18.75    62.70  29.42

         DiabetesPedigreeFunction    Age
Cluster
0                            0.51  41.12
1                            0.44  26.61
```

```
plt.figure(figsize=(12, 6))

sns.heatmap(
    cluster_means,
    annot=True,
    fmt='.2f',
    cmap='coolwarm'
)

plt.title("Average Feature Values Across Clusters")
plt.xlabel("Features")
plt.ylabel("Cluster")
plt.show()
```



```
[ ]
print("========== FINAL RESULTS ==========")

print("\nNumber of clusters:", 2)

print("\nCluster Size:")
print(df['Cluster'].value_counts().sort_index())

print("\nSilhouette Score:")
print(round(sil_score, 4))

print("\nPCA Explained Variance:")
print(pca.explained_variance_ratio_.round(4))

print("\nTotal Variance Explained by 2 PCs:")
print(round(pca.explained_variance_ratio_.sum(), 4))

print("\nCluster vs Outcome:")
print(pd.crosstab(df['Cluster'], df['Outcome']))
```

```
========== FINAL RESULTS ==========

Number of clusters: 2

Cluster Size:
Cluster
0    351
1    417
Name: count, dtype: int64

Silhouette Score:
0.1963

PCA Explained Variance:
[0.2854 0.1869]

Total Variance Explained by 2 PCs:
0.4723

Cluster vs Outcome:
Outcome    0    1
Cluster
0        156  195
1        344   73
```

Conclusion

Clustering and dimensionality reduction techniques were applied to the Pima Indians Diabetes Dataset. K-Means clustering was used to group patients based on similarities in their medical attributes, while PCA was used to reduce the dimensionality of the dataset for visualization. The resulting clusters were evaluated and interpreted using appropriate measures and visualizations.

Questions to be Answered by Students

1. What value of *k*was selected using the Elbow Method, and why?

ANS : The value of k = 2 was selected for K-Means clustering. It was chosen based on the Elbow Method, where increasing the number of clusters beyond this point provided comparatively smaller improvement in WCSS.

2. What Silhouette Score was obtained, and what does it indicate about the quality of the clusters?

ANS : The obtained Silhouette Score was [0.19627861483317974]. It indicates how well the observations are separated within their assigned clusters. A higher positive value indicates better-defined and more separated clusters.

3. Which variables contributed most to the first two principal components?

ANS : The variables with the largest absolute PCA loadings contributed most to the first two principal components. For the Pima dataset, variables such as Glucose, BMI, Age, and Pregnancies generally contribute strongly, but the exact variables should be identified from the PCA loadings obtained in the experiment.

4. What patterns were observed in the PCA visualization of the clusters?

ANS : The PCA visualization showed that the observations formed two broad groups corresponding to the K-Means clusters. Some separation between the clusters was visible, although there was also considerable overlap, indicating that the patient groups are not completely distinct.

5. How do the discovered clusters compare with the known Outcome categories?

ANS : The discovered clusters showed some relationship with the known outcome categories, but they did not exactly match them. This is expected because K-Means clustering was performed without using outcome. Therefore, the clusters represent similarities in the medical features rather than directly representing diabetic and non-diabetic categories.